

Specialized Configurations with Docker

Environment Variables (OCI Spec)

You can control the behavior of the NVIDIA Container Runtime using environment variables, especially for enumerating the GPUs and the capabilities of the driver. Each environment variable maps to a command-line argument for `nvidia-container-cli` from [libnvidia-container](#). These variables are already set in the NVIDIA-provided base [CUDA images](#).

GPU Enumeration

You can specify GPUs to the Docker CLI using either the `--gpus` option starting with Docker [19.03](#) or the environment variable `NVIDIA_VISIBLE_DEVICES`. This variable controls which GPUs are accessible inside the container.

The possible values of the `NVIDIA_VISIBLE_DEVICES` variable are:

Possible values	Description
<code>0,1,2,</code> or <code>GPU-fef8089b</code>	a comma-separated list of GPU UUID(s) or index(es).
<code>all</code>	All GPUs are accessible. This is the default value in base CUDA container images.
<code>none</code>	No GPU is accessible, but driver capabilities are enabled.
<code>void</code> or <i>empty</i> or <i>unset</i>	<code>nvidia-container-runtime</code> has the same behavior as <code>runc</code> (that is, neither GPUs nor capabilities are exposed).

 Note

When using the `--gpus` option to specify the GPUs, use the `device` parameter, as shown in the following examples. Encapsulate the format of the `device` parameter within single quotes, followed by double quotes for the devices you want enumerated to the container. For example, `'"device=2,3"'` enumerates GPUs 2 and 3 to the container.

When using the `NVIDIA_VISIBLE_DEVICES` variable, you might need to set `--runtime` to `nvidia` unless it is already set as the default.

The following examples show common usage:

1. Start a GPU-enabled CUDA container using `--gpus`:

```
$ docker run --rm --gpus all nvidia/cuda nvidia-smi
```

2. Use `NVIDIA_VISIBLE_DEVICES` and specify the NVIDIA runtime:

```
$ docker run --rm --runtime=nvidia \  
-e NVIDIA_VISIBLE_DEVICES=all nvidia/cuda nvidia-smi
```

3. Start a GPU-enabled container on two GPUs:

```
$ docker run --rm --gpus 2 nvidia/cuda nvidia-smi
```

4. Start a GPU-enabled container on specific GPUs:

```
$ docker run --gpus '"device=1,2"' \  
nvidia/cuda nvidia-smi --query-gpu=uuid --format=csv
```

```
uuid  
GPU-ad2367dd-a40e-6b86-6fc3-c44a2cc92c7e  
GPU-16a23983-e73e-0945-2095-cdeb50696982
```

5. Alternatively, use `NVIDIA_VISIBLE_DEVICES`:

```
$ docker run --rm --runtime=nvidia \  
-e NVIDIA_VISIBLE_DEVICES=1,2 \  
nvidia/cuda nvidia-smi --query-gpu=uuid --format=csv
```

```
uuid  
GPU-ad2367dd-a40e-6b86-6fc3-c44a2cc92c7e  
GPU-16a23983-e73e-0945-2095-cdeb50696982
```

6. Query the GPU UUID using `nvidia-smi`, then specify it to the container:

```
$ nvidia-smi -i 3 --query-gpu=uuid --format=csv
```

```
uuid  
GPU-18a3e86f-4c0e-cd9f-59c3-55488c4b0c24
```

```
$ docker run --gpus device=GPU-18a3e86f-4c0e-cd9f-59c3-55488c4b0c24 \  
nvidia/cuda nvidia-smi
```

Requesting IMEX Channels

Use the `NVIDIA_IMEX_CHANNELS` environment variable to request NVIDIA IMEX channels for a container. Specify one or more numeric channel IDs as a comma-separated list.

The following command requests channels 0 and 1:

```
$ docker run --rm --runtime=nvidia \
  -e NVIDIA_VISIBLE_DEVICES=all \
  -e NVIDIA_IMEX_CHANNELS=0,1 \
  <image> <command>
```

In CDI and JIT-CDI mode, each channel ID must meet both requirements:

- The ID is in the range from 0 through 1,048,575.
- The corresponding `/dev/nvidia-caps-imex-channels/channel<ID>` device exists on the host.

If either requirement is not met, container creation fails with an error that identifies the invalid or missing channel. Inspect `/dev/nvidia-caps-imex-channels/` on the host and request only the channel IDs that are present.

Driver Capabilities

The `NVIDIA_DRIVER_CAPABILITIES` variable controls which driver libraries and binaries are mounted inside the container.

The possible values of the `NVIDIA_DRIVER_CAPABILITIES` variable are:

Possible values	Description
<code>compute,video</code> or <code>graphics,utility</code>	a comma-separated list of driver features the container needs.
<code>all</code>	enable all available driver capabilities.
<code>empty</code> or <code>unset</code>	use default driver capability: <code>utility</code> , <code>compute</code>

The following table describes the supported driver capabilities:

Driver Capability	Description
<code>compute</code>	required for CUDA and OpenCL applications. When present on the host, the NVIDIA OpenCL ICD file is also available in the container.
<code>compat32</code>	required for running 32-bit applications.

Driver Capability	Description
<code>graphics</code>	required for rendering OpenGL, EGL, and Vulkan applications.
<code>utility</code>	required for using <code>nvidia-smi</code> and NVML.
<code>video</code>	required for using the Video Codec SDK.
<code>display</code>	required for displaying X11 or Wayland windows. Implies <code>graphics</code> .

Note

`NVIDIA_DRIVER_CAPABILITIES` replaces the default capabilities rather than adding to them, so list every capability your application needs. An application that displays 3D rendered output on X11 or Wayland requires `compute,utility,graphics,display`, for instance.

For example, to allow usage of CUDA and NVML, specify the `compute` and `utility` capabilities:

```
$ docker run --rm --runtime=nvidia \
  -e NVIDIA_VISIBLE_DEVICES=2,3 \
  -e NVIDIA_DRIVER_CAPABILITIES=compute,utility \
  nvidia/cuda nvidia-smi
```

```
$ docker run --rm --gpus 'all,"capabilities=compute,utility"' \
  nvidia/cuda:12.5.0-base-ubuntu22.04 nvidia-smi
```

To run an OpenGL, EGL, or Vulkan application on a selected GPU, include the `graphics` capability. The following command makes GPU 0 and the graphics and utility driver components available to the container:

```
$ docker run --rm --runtime=nvidia \
  -e NVIDIA_VISIBLE_DEVICES=0 \
  -e NVIDIA_DRIVER_CAPABILITIES=graphics,utility \
  <image> <command>
```

Legacy mode limits EGL and Vulkan visibility to the physical GPUs assigned to a container when the `graphics` or `display` driver capability is enabled. The 1.20.0 release adds the same behavior to CDI and JIT-CDI modes through the `update-application-profile` hook. This change provides feature parity across the modes.

If the application-profile hook conflicts with an application, refer to [Disabling Hooks in JIT-CDI Mode](#).

Constraints

The NVIDIA runtime also lets you define constraints on the configurations that the container supports.

NVIDIA_REQUIRE_* Constraints

This variable is a logical expression to define constraints on the software versions or GPU architectures on the container.

The following table describes the supported constraints:

Constraint	Description
<code>cuda</code>	constraint on the CUDA driver version.
<code>driver</code>	constraint on the driver version.
<code>arch</code>	constraint on the compute architectures of the selected GPUs.
<code>brand</code>	constraint on the brand of the selected GPUs (such as GeForce, Tesla, GRID).

Multiple constraints can be expressed in a single environment variable: space-separated constraints are ORed, comma-separated constraints are ANDed. Multiple environment variables of the form `NVIDIA_REQUIRE_*` are ANDed together.

For example, the following constraints can be specified to the container image for constraining the supported CUDA and driver versions:

```
NVIDIA_REQUIRE_CUDA "cuda>=11.0 driver>=450"
```

NVIDIA_DISABLE_REQUIRE Environment Variable

Single switch to disable all the constraints of the form `NVIDIA_REQUIRE_*`.

 **Note**

If you are running CUDA base images older than CUDA 11.7 and cannot update to the new base images with updated constraints, you can disable CUDA compatibility checks by setting `NVIDIA_DISABLE_REQUIRE` to `true`.

NVIDIA_REQUIRE_CUDA Constraint

The version of the CUDA toolkit used by the container. It is an instance of the generic `NVIDIA_REQUIRE_*` case and it is set by official CUDA images. If the version of the NVIDIA driver is insufficient to run this version of CUDA, the container does not start. This variable can be specified in the form `major.minor`.

The possible values for this variable are `cuda>=7.5`, `cuda>=8.0`, `cuda>=9.0`, and so on.

Dockerfiles

You can set capabilities and GPU enumeration in images using environment variables. If you set the environment variables inside the Dockerfile, you do not need to set them on the `docker run` command line.

For instance, if you are creating your own custom CUDA container, you should use the following:

```
ENV NVIDIA_VISIBLE_DEVICES all
ENV NVIDIA_DRIVER_CAPABILITIES compute,utility
```

These environment variables are already set in the NVIDIA-provided CUDA images.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2020-2026, NVIDIA Corporation.